

E-COMMERCE PROJECT

1. Product Overview

Product Name

ShopSphere (working name)

Vision

A multi-tenant, modern e-commerce platform enabling users to browse, purchase, and track products with a seamless UX, and enabling admins/sellers to manage catalog, inventory, and orders at scale.

Core Principles

- High availability (production-grade)
 - Horizontal scalability
 - Secure by default
 - API-first design
 - Mobile-first UX
 - Modular architecture (microservices-ready evolution)
-

2. User Roles

1. Customer (Buyer)

- Browse products
- Search & filter
- Add to cart / wishlist
- Checkout & payment
- Track orders
- Manage profile

2. Admin

- Full system control
- Manage users
- Manage products/categories
- Manage orders & refunds
- Analytics dashboard

3. Seller (optional multi-vendor extension)

- Add/manage products
 - Manage inventory
 - View orders
 - Manage fulfillment
-

3. Functional Requirements

3.1 Authentication & Authorization

- Email/password signup & login
 - JWT-based authentication (access + refresh tokens)
 - Role-based access control (RBAC)
 - OAuth support (Google login)
 - Password reset via email OTP/link
 - Account verification (email verification mandatory)
-

3.2 Product Management

- Create/update/delete products (admin/seller)
- Product fields:
 - title, description, images (multi)
 - price, discount, tax

- category, subcategory
 - SKU
 - stock quantity
 - ratings & reviews
 - Product variants (size, color, etc.)
 - Bulk upload via CSV
-

3.3 Catalog & Search

- Product listing page
 - Filters:
 - price range
 - category
 - rating
 - availability
 - Sorting:
 - price low/high
 - popularity
 - newest
 - Full-text search (Elasticsearch or MongoDB Atlas Search)
-

3.4 Cart System

- Add/remove items
 - Update quantity
 - Persistent cart (guest + logged-in merge)
 - Price recalculation in real-time
 - Stock validation before checkout
-

3.5 Checkout Flow

- Address management (CRUD)
 - Order summary page
 - Coupon/discount system
 - Tax + shipping calculation
 - Payment integration:
 - Razorpay / Stripe (preferred abstraction layer)
-

3.6 Order Management

- Order creation after payment
 - Order statuses:
 - pending → paid → processing → shipped → delivered → cancelled → refunded
 - Order history for users
 - Admin order control panel
-

3.7 Payment System

- Secure payment gateway integration
 - Webhook handling (critical)
 - Payment verification (server-side only)
 - Refund support
-

3.8 Reviews & Ratings

- Users can rate products (1–5)
 - Write reviews (text + optional images)
 - Verified purchase badge logic
 - Admin moderation system
-

3.9 Notifications System

- Email notifications:
 - order confirmation
 - shipping updates
 - Optional SMS/WhatsApp integration
 - In-app notifications
-

3.10 Admin Dashboard

- Revenue analytics
 - Order tracking
 - User management
 - Product performance metrics
 - Sales charts (daily/monthly)
 - Low-stock alerts
-

4. Non-Functional Requirements (Critical)

4.1 Performance

- API response time: < 200–300ms (95th percentile)
 - Page load: < 2.5s (LCP optimized)
 - CDN for images (Cloudinary/S3 + CDN)
 - Lazy loading & pagination mandatory
-

4.2 Scalability

- Stateless backend (horizontal scaling ready)
- Separate services/modules:
 - auth service
 - product service
 - order service

- payment service
 - Redis caching layer for:
 - sessions
 - product cache
 - cart data
-

4.3 Security

- JWT + refresh token rotation
 - HTTP-only cookies (no localStorage tokens for production)
 - Input validation (Zod/Joi)
 - Rate limiting (brute force protection)
 - Helmet + CORS strict configuration
 - SQL/NoSQL injection protection
 - XSS prevention
 - Webhook signature verification (payments)
 - RBAC enforcement middleware
-

4.4 Reliability

- Graceful error handling
 - Centralized logging (Winston / Pino)
 - Retry mechanisms for external APIs
 - Queue system for async tasks (BullMQ / RabbitMQ)
 - Idempotency for payment/order creation
-

4.5 Availability

- Target uptime: 99.9%
 - Fail-safe payment flow
 - Database replication + backups
-

4.6 Maintainability

- Clean architecture (controller-service-repository pattern)
 - Modular folder structure
 - Type safety (TypeScript mandatory)
 - Linting + formatting (ESLint + Prettier)
 - API documentation (Swagger/OpenAPI)
-

4.7 Observability

- Request logging
 - Error tracking (Sentry)
 - Metrics (Prometheus/Grafana optional)
 - Audit logs for admin actions
-

5. Suggested System Architecture (MERN-based)

Frontend

- React (Next.js preferred for SEO)
- Redux Toolkit / Zustand (state management)
- React Query (server state)
- TailwindCSS / ShadCN UI

Backend

- Node.js + Express / NestJS (preferred for scalability)
- TypeScript mandatory
- REST APIs (GraphQL optional future upgrade)

Database

- MongoDB (primary)
- Redis (cache + sessions + queues)

Storage

- AWS S3 / Cloudinary for images

Infra (future-ready)

- Dockerized services
 - Nginx reverse proxy
 - CI/CD (GitHub Actions)
 - Deploy: AWS / Vercel / Render (staged)
-

6. Core Database Entities (High-Level)

- User
 - Product
 - Category
 - Cart
 - Order
 - OrderItem
 - Payment
 - Review
 - Coupon
 - Address
 - Inventory
-

7. API Design Standards

- RESTful naming conventions
- Versioning: `/api/v1`

- Pagination: `limit + offset/cursor`
- Consistent response format:

```
{  
  "success":true,  
  "data": {},  
  "message": "",  
  "error":null  
}
```

8. Edge Cases You MUST Handle

- Payment success but order failure (reconciliation needed)
- Duplicate webhook calls (idempotency key)
- Out-of-stock during checkout
- Race condition in inventory updates
- Guest cart → login cart merge conflicts
- Token expiration mid-session
- Partial failures in order pipeline

9. Delivery Expectations

You will deliver:

Phase 1

- Architecture + DB design
- Auth system

Phase 2

- Product + catalog system
- Cart system

Phase 3

- Checkout + payment integration

Phase 4

- Orders + admin panel

Phase 5

- Optimization + security hardening + deployment
-

10. Final Engineering Expectation

This is not a CRUD project.

This is expected to be:

- production-grade
- scalable design
- clean architecture
- secure payment flow
- real-world failure handling
- industry-level code structure